	**Fundamentos de Sistemas Embebidos**
Facultad de Ingeniería	UNAM

Laboratorio de Fundamentos de Sistemas Embebidos

Profesor(a): Eduardo Fragoso Navarro

Asignatura: Fundamentos de Sistemas Embebidos

Grupo: 3

No de Práctica(s): 8

Integrante(s): Tepal Briseño Hansel Yael - 320078282

Ugartechea González Luis Antonio - 320223822

No. de brigada: 12

Semestre: 9°

Fecha de entrega: 3 de noviembre de 2026

Observaciones:

CALIFICACIÓN: _____

Node-RED: control de un programa de Python por SSH

Objetivo

El objetivo principal de esta práctica consiste en instalar e implementar Node-RED mediante un contenedor de Docker dentro de una placa Orange Pi Zero 2W, explorando las ventajas del paradigma de la programación low-code orientada a dispositivos IoT. A través de esta plataforma, se busca establecer una interfaz de control visual basada en flujos que permita gestionar la ejecución de procesos en el hardware, facilitando el prototipado rápido sin la necesidad de programar toda la lógica de integración de manera manual.

De manera específica, se pretende diseñar un flujo capaz de arrancar, detener y consultar el estado de un script de Python (`2_puntos.py`), el cual está encargado de generar gráficos en movimiento dentro de una pantalla ILI9341 conectada a la placa. Para lograr el control de este proceso, el contenedor de Node-RED establecerá una conexión segura hacia la placa a través de SSH mediante el uso de llaves públicas y privadas, ejecutando comandos remotos de systemd para modificar el estado del servicio del programa de forma automatizada.

Resultados y Análisis

Ejercicio 1: Instalar Node-RED con Docker

En el primer ejercicio se llevó a cabo la instalación y despliegue de Node-RED utilizando un contenedor de Docker en la Orange Pi Zero 2W. Como resultado, se logró levantar el contenedor exitosamente en segundo plano y se comprobó su estado activo mediante el comando `docker ps`. A continuación, se muestra la salida de la terminal confirmando que el contenedor `nodered` se encuentra en ejecución:

```
orangepi@equipo12:~$ docker ps
CONTAINER ID   IMAGE              COMMAND             CREATED      STATUS                          PORTS                                                NAMES
b31b98dfbb7b   nodered/node-red   "./entrypoint.sh"   5 days ago   Up 13 seconds (health: starting)   0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp   nodered
orangepi@equipo12:~$ hostname -I
192.168.0.105 172.17.0.1 fd65:e6df:2a53:75b2:3e0b:40ff:fe21:1251
orangepi@equipo12:~$
```

Figure 1: Salida de la terminal mostrando el contenedor de Node-RED en ejecución.

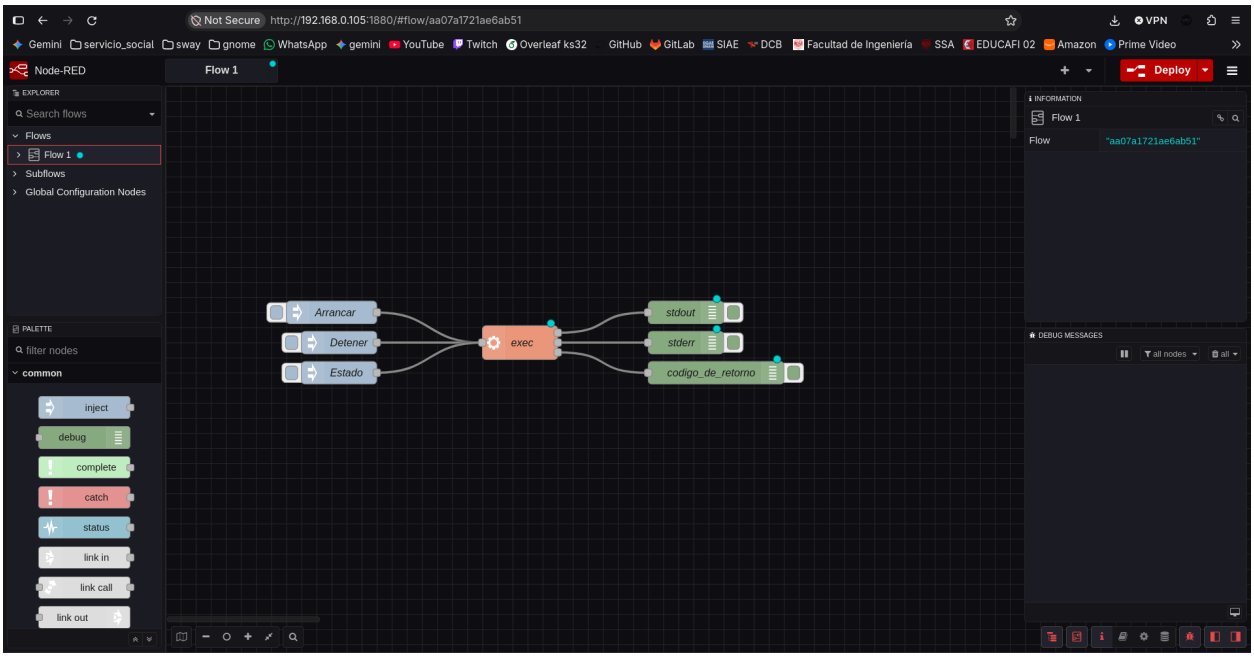


Figure 2: Interfaz web de Node-RED accesible desde el navegador de la PC.

Es importante mencionar la configuración de la carpeta persistente montada como volumen a través del parámetro `-v "$PWD/data":/data`. Dado que los contenedores son momentáneos por naturaleza, este mapeo hacia el sistema de archivos de la placa garantiza que toda la información crítica de Node-RED —tales como los flujos de trabajo diseñados y las futuras llaves SSH— no se pierda al detener o eliminar el contenedor. Asimismo, la asignación de permisos adecuados para el usuario `uid 1000` (correspondiente al usuario `orangepi`) resultó indispensable para evitar errores de escritura al momento en que el contenedor intentara guardar datos en el directorio local de la placa. Finalmente, el mapeo de puertos (`-p 1880:1880`) fue lo que permitió exponer la aplicación aislada hacia la red local, habilitando su control externo.

Ejercicio 2: Dejar listo el servicio puntos

En el segundo ejercicio se preparó el servicio `puntos` para que su ejecución sea controlada exclusivamente por Node-RED, evitando que arranque de manera automática al encender la placa. Para lograr esto, se deshabilitó y detuvo el servicio utilizando los comandos `systemctl -user disable puntos` y `systemctl -user stop puntos`. Posteriormente, se verificó su estado para confirmar que el proceso se encontrara inactivo.

A continuación, se muestra la salida en la terminal comprobando que el servicio fue detenido correctamente:

```
orangepi@equipo12:~$ systemctl --user disable puntos
orangepi@equipo12:~$ systemctl --user stop puntos
orangepi@equipo12:~$ systemctl --user status puntos --no-pager
○ puntos.service - Puntos ILI9341
     Loaded: loaded (/home/orangepi/.config/systemd/user/puntos.service; disabled; preset: enabled)
     Active: inactive (dead)
orangepi@equipo12:~$
```

Figure 3: Salida de la terminal mostrando el servicio `puntos` detenido.

Una vez garantizado que el servicio no iniciaba por sí solo, se realizó una prueba manual arrancando y deteniendo el servicio (`systemctl -user start puntos` y `systemctl -user stop puntos`) para comprobar que el hardware respondía correctaser evide mente antes de involucrar a Node-RED.

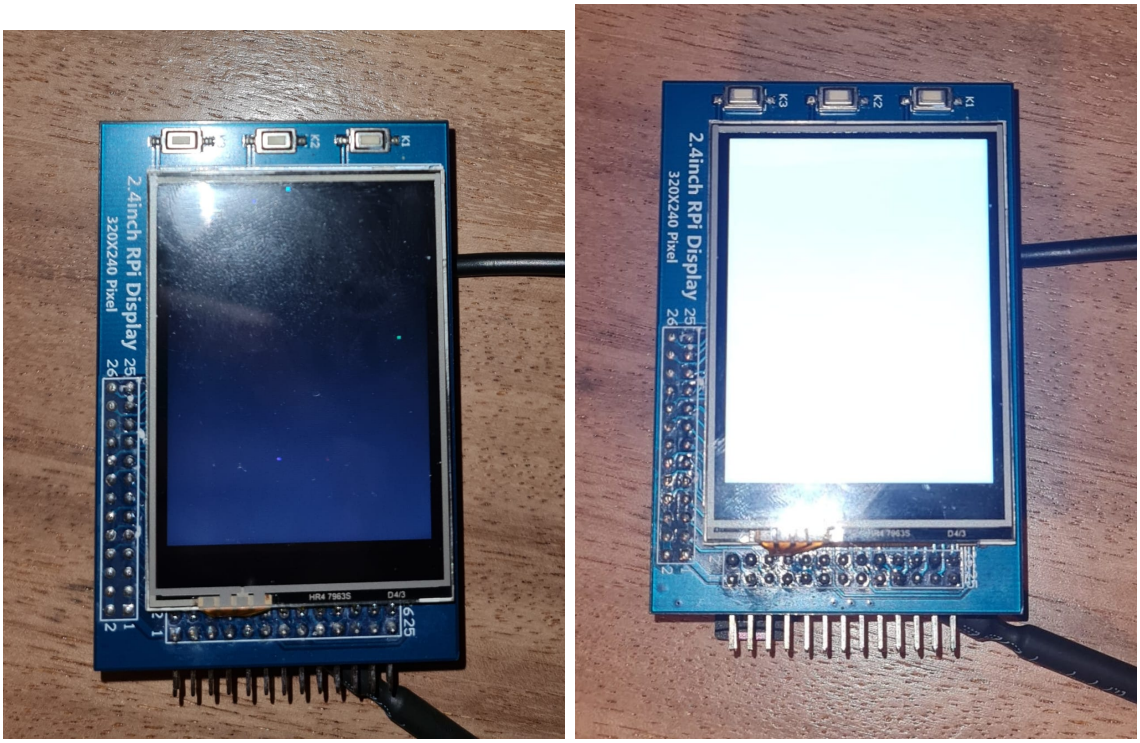


Figure 4: Prueba del servicio mostrando los puntos en la pantalla física ILI9341.

Aquí notamos la importancia de aislar las capas de control en el desarrollo con sistemas embebidos. Al evitar que el programa inicie con el sistema operativo, se previenen bloqueos de acceso al hardware, de lo contrario, si Node-RED intentara iniciar un proceso que ya está usando la pantalla, se generaría un error de colisión de recursos (*Device or resource busy*) debido a que la interfaz no soporta múltiples programas escribiendo simultáneamente. Además, realizar la prueba manual de `systemd` actúa como un punto de control fundamental: permite validar que el script de Python y el bus de datos hacia la pantalla funcionan correctamente por sí solos. De esta forma, si ocurren

fallos en los siguientes ejercicios, el diagnóstico será más sencillo al saber que el problema radicaría en la conexión SSH o en la lógica del flujo, y no en la ejecución local del programa.

Ejercicio 3: SSH sin contraseña desde el contenedor

En el tercer ejercicio, se configuró y verificó el acceso remoto mediante SSH sin contraseña desde el interior del contenedor de Node-RED hacia el sistema operativo de la placa. Dado que las llaves criptográficas (pública y privada) ya habían sido generadas previamente dentro de la carpeta persistente y la llave pública ya constaba en el archivo `~/.ssh/authorized_keys` del usuario anfitrión, la actividad se centró en comprobar la persistencia de esta configuración y validar la comunicación.

Para confirmar que el enlace seguro seguía habilitado, se ejecutó un comando de prueba desde el contenedor solicitando a la placa que imprimiera un mensaje, obteniendo una respuesta exitosa sin que el sistema solicitara credenciales interactivas:

```
orangepi@equipo12:~$ docker exec nodered ssh -i /data/.ssh/id_ed25519 -o StrictHostKeyChecking=no orangepi@192.168.0.105 echo ssh_sin_contraseña
Warning: Permanently added '192.168.0.105' (ED25519) to the list of known hosts.
ssh_sin_contraseña
```

Figure 5: Prueba de conexión SSH desde el contenedor hacia la placa.

Posteriormente, se comprobó que, a través de esta misma conexión SSH, el contenedor contaba con los privilegios necesarios para interactuar con el administrador de servicios `systemd` del usuario en la placa base, enviando una consulta remota sobre el estado del servicio de los puntos:

```
orangepi@equipo12:~$ docker exec nodered ssh -i /data/.ssh/id_ed25519 -o StrictHostKeyChecking=no orangepi@192.168.0.105 systemctl --user is-active puntos
inactive
orangepi@equipo12:~$
```

Figure 6: Prueba de control remoto de `systemd` desde el contenedor.

Se observa la eficiencia de combinar volúmenes de Docker con autenticación asimétrica. Al tener la llave privada alojada en el directorio montado `/data/.ssh`, la identidad del contenedor sobrevive a reinicios, permitiendo la automatización sostenida. Asimismo, el uso de los parámetros `-i` (para apuntar a la ruta exacta de la llave privada) y `-o StrictHostKeyChecking=no` resulta vital para este tipo de arquitecturas IoT; puesto que Node-RED opera en segundo plano como un servicio desatendido, carece de una terminal interactiva para aceptar huellas criptográficas (*fingerprints*) de nuevos dispositivos o para teclear contraseñas. Esta delegación de permisos confirma que Node-RED puede solicitar acciones en el hardware subyacente manteniendo las barreras de seguridad del contenedor intactas.

Ejercicio 4: El flujo en Node-RED

En el último ejercicio se comprobó el funcionamiento del sistema utilizando la interfaz gráfica de Node-RED. Dado que la configuración de los nodos (tres nodos `inject`, un nodo `exec` y tres nodos `debug`) y la conexión SSH ya estaban establecidas, el objetivo fue validar que los comandos enviados desde la interfaz web lograran arrancar, detener y consultar el estado del servicio `puntos` de manera efectiva.

Al desplegar (*Deploy*) el flujo y abrir el panel de mensajes (*debug*) en Node-RED, se procedió a interactuar con los nodos `inject`. Primero, se comprobó el estado inactivo del servicio presionando el inyector correspondiente, lo cual generó un mensaje `"inactive\n"` en la salida estándar (debug 1) y un código de error normal en la salida de retorno (debug 3), confirmando que `systemd` reconocía el servicio pero este no estaba corriendo.

A continuación, se presenta el diagrama completo del flujo implementado en Node-RED:

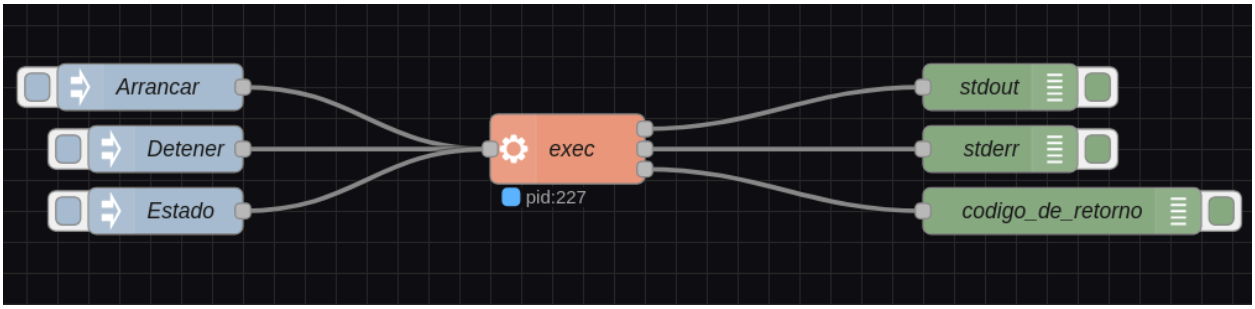


Figure 7: Diagrama del flujo en Node-RED para el control del servicio `puntos`.

Posteriormente, se inyectó el comando de arranque (`start puntos`). Esto se reflejó inmediatamente en la ejecución del script en la placa, donde los gráficos comenzaron a renderizarse de manera fluida en la pantalla ILI9341. El panel de depuración mostró un código de retorno `0` en debug 3, indicando una ejecución remota exitosa sin errores en stderr (debug 2). Al volver a consultar el estado, la salida cambió exitosamente a `active`. Finalmente, se activó el inyector de detención (`stop puntos`), logrando detener la animación instantáneamente, comprobando así el control bidireccional asíncrono entre el entorno *low-code* y el *hardware* embebido.

Para respaldar el análisis, se capturó la salida del panel debug durante las transiciones de estado:

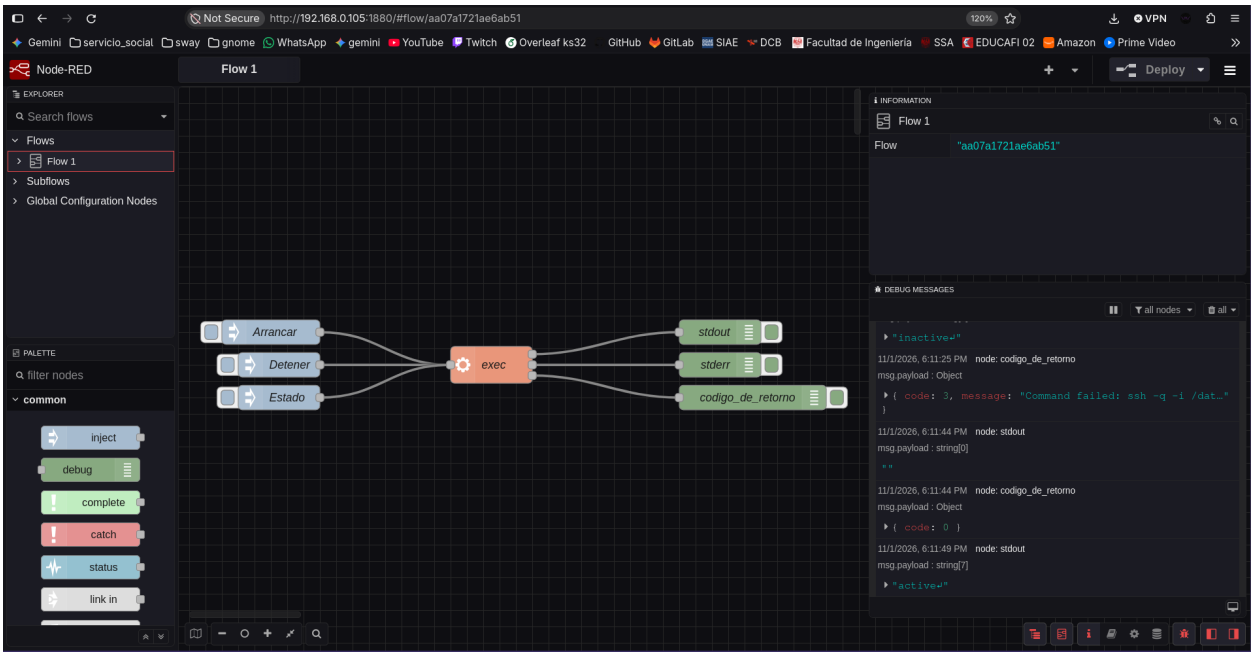


Figure 8: Panel debug mostrando las respuestas de `systemctl` ante los comandos inyectados.

Esta implementación final subraya el valor de la programación *low-code* en entornos IoT. Al encapsular la complejidad de las conexiones SSH y las llamadas al sistema en bloques visuales preconfigurados, se reduce significativamente la fricción operativa. Aunque el desarrollo inicial requiere conocimientos técnicos para configurar las llaves y contenedores, una vez establecido, permite que operadores sin experiencia profunda en administración de sistemas Linux puedan integrar, monitorear y controlar el hardware mediante interfaces gráficas intuitivas.

Conclusiones

Después de hacer esta práctica, queda clara la diferencia entre controlar servicios con Node-RED y hacerlo con programación tradicional (como armar un script en Python o Bash). Armar la lógica en Node-RED fue mucho más rápido y visual, ya que ver los bloques te permite entender todo el flujo de datos a simple vista. En cambio, hacerlo de forma tradicional nos habría obligado a escribir todo el código para manejar la conexión SSH, capturar los errores y procesar los comandos a mano. Curiosamente, lo que más tiempo nos tomó no fue armar el flujo en sí, sino toda la configuración inicial de permisos y llaves.

Por otro lado, aunque el flujo es totalmente visual, el hecho de tener que usar la terminal para instalar Docker, generar las llaves SSH y preparar `systemd` nos dice mucho sobre los límites del *low-code*. Estas herramientas son una maravilla para conectar sistemas, sensores y hardware de forma rápida, pero siguen dependiendo por completo de que el sistema operativo esté bien configurado. En resumen: el *low-code* te ahorra escribir la lógica de la aplicación, pero no te

salva de tener que saber usar Linux.

Yo usaría Node-RED principalmente en proyectos de IoT, automatización de hardware o cuando se necesita sacar un prototipo rápido; sobre todo si en el equipo hay personas de otras áreas (como mantenimiento u operaciones) que necesitan entender o modificar el sistema sin ser expertos en código. Preferiría la programación tradicional para proyectos que exijan mucho rendimiento, donde sea clave llevar un control de versiones estricto con Git (ya que manejar versiones de texto es más fácil que rastrear cambios en los archivos de Node-RED) o en sistemas muy grandes que dependan de pruebas automatizadas.